SDG BLOCKCHAIN ACCELERATOR

PoC Report
TradeChainX & UNDP Ethiopia

Project Name: Blockchain-Based Carbon Credits MRV Platform
Challenge & UNDP Office: UNDP Ethiopia
Document Version: v1.1

2. Project Overview

This prototype demonstrates a blockchain-based implementation of a **Measurement, Reporting, and Verification (MRV) system** for carbon credit projects, built on the Cardano network using the eUTxO model.

The system is designed to digitize and enforce the lifecycle of carbon credit generation through a combination of on-chain validators and off-chain infrastructure. Each project progresses through a defined lifecycle, registration, reporting, verification, and issuance, where state transitions are enforced via smart contracts.

Rather than storing raw MRV data on-chain, the prototype follows a hybrid approach:

- **Off-chain systems (Supabase + IPFS)** handle structured data and large files
- **On-chain contracts** enforce state integrity and token issuance rules

The prototype successfully demonstrates:

- Creation of project state on-chain
- Controlled verification flow through authorized actors
- Minting of carbon credit tokens based on verified data
- Transfer and retirement of tokens through a marketplace flow

3. Repository Structure

```
/prototype
│
├── /contracts
│   ├── project_registry.ak        # Initializes project UTxO
│   ├── lifecycle_validator.ak     # Controls state transitions
│   ├── verification.ak            # Verifier logic
│   ├── minting_policy.ak          # Token mint/burn logic
│
├── /tests
│   ├── unit/                      # Validator unit tests
```

```
│   ├── emulator/                 # End-to-end flows
│
├── /scripts
│   ├── deploy.ts                 # Contract deployment
│   ├── register_project.ts       # Registration flow
│   ├── verify_project.ts         # Verification flow
│   ├── mint_credits.ts           # Minting logic
│   ├── retire_credits.ts         # Burn logic
│
├── /frontend
│   ├── src/
│   │   ├── components/           # UI components
│   │   ├── pages/                # Dashboard + flows
│   │   ├── wallet/               # CIP-30 integration
│
├── /backend
│   ├── services/
│   │   ├── mrv.service.ts        # MRV processing
│   │   ├── verifier.service.ts   # Role management
│   ├── db/
│   │   ├── supabase.client.ts
│
├── /docs
│   ├── README.md
│   ├── setup.md
│   ├── architecture.md
│   ├── screenshots/
```

4. Build Instructions

4.1 Prerequisites

- Aiken CLI (latest stable)
- Node.js ≥ 18
- Cardano wallet (Nami / Lace)
- Blockfrost API key
- Supabase project instance

4.2 Environment Configuration

```
CARDANO_NETWORK=preprod
BLOCKFROST_API_KEY=xxxx

SUPABASE_URL=xxxx
SUPABASE_ANON_KEY=xxxx

IPFS_API_KEY=pinata_key
```

4.3 Build Process

Compile Smart Contracts

```
aiken build
```

Install Backend / Frontend

```
npm install
```

Start Services

```
npm run dev
```

4.4 Output Artifacts

- Compiled Plutus scripts
- Validator addresses
- Policy IDs
- Deployment logs

5. Test Instructions

5.1 Test Strategy

Testing covers:

- Unit-level validation logic
- End-to-end lifecycle flows
- Failure scenarios and edge cases

5.2 Unit Tests

aiken test

Coverage includes:

- Signature validation (owner / verifier)
- Datum structure validation
- Redeemer matching logic

5.3 Emulator Tests

End-to-end flows executed using Aiken emulator:

aiken check

5.4 Test Scenarios

Scenario	Expected Result
Valid registration	UTxO created
Unauthorized verification	Rejected
Invalid mint attempt	Fails
Correct minting	Tokens issued
Retirement	Tokens burned

5.5 Edge Cases Covered

- Duplicate mint attempts
- Invalid verifier address
- Incorrect state transitions
- Missing metadata hash
- Replay transaction attempts

6. Deployment Instructions

6.1 Deployment Flow

```
[1] Compile contracts (Aiken)
[2] Generate validator addresses
[3] Deploy reference scripts (optional)
[4] Create initial UTxOs
[5] Register project
```

6.2 Deployment Steps

1. Build contracts:

```
aiken build
```

2. Initialize Lucid:

```
lucid.selectWallet(...)
```

3. Deploy validators:
- Project registry
- Lifecycle validator
- Minting policy
4. Submit transactions via Blockfrost

6.3 Deployed Components

Component	Status
Project Registry	Deployed
Lifecycle Validator	Deployed
Minting Policy	Deployed
Marketplace	Partial (prototype level)

7. Prototype Capabilities

7.1 Implemented

- Project registration on-chain
- MRV data submission (off-chain)
- Verification transaction flow
- Token minting tied to verification
- Token transfer between wallets
- Token retirement (burn mechanism)

7.2 Partially Implemented

- Marketplace UI (basic flow only)
- Verifier management (manual assignment)

7.3 Not Implemented

- Automated MRV data ingestion
- Oracle integration
- Governance layer

8. Dependencies & Environment

Category	Tool
Smart Contracts	Aiken
Blockchain	Cardano Preprod
Tx Layer	Lucid
API	Blockfrost
Backend	Node.js
Database	Supabase
Storage	IPFS (Pinata)
Frontend	React

9. Technical Limitations

- MRV validation remains partially manual
- No automated oracle data feeds
- Limited UI/UX for marketplace flows
- Execution unit optimization not finalized
- No formal audit performed

10. Remaining Issues / Next Steps

10.1 Smart Contract Improvements

- Optimize execution units
- Reduce datum size
- Improve validation efficiency

10.2 Security

- Conduct external audit
- Introduce verifier registry / access control

10.3 MRV Automation

- Integrate IoT or satellite data
- Introduce oracle-based verification triggers

10.4 Productization

- Improve frontend UX
- Expand marketplace features
- Add analytics dashboard

10.5 Deployment Readiness

- Prepare for mainnet deployment
- Align with carbon standards (Verra, Gold Standard)

11. Summary

The prototype successfully demonstrates the feasibility of implementing a carbon credit MRV system on Cardano using a hybrid architecture. While core flows, registration, verification, minting, and retirement, are functional, further work is required to improve automation, security, and scalability before production deployment.